AIBO Histogram and EM Notes for Forth

Summary This note recasts the relevant parts of *AIBO's First Words* as a Forth-facing implementation guide. Instead of treating the paper as a Java-adjacent reference, it describes both the page 17 histogram matcher and the appendix EM clustering algorithm as arrays, scalar parameters, and stack-oriented words. The goal is to support a future Forth module for AIBO-related experimentation without depending on Java source code from the *Data Mining* reference.

Purpose

The paper cites Ian H. Witten and Eibe Frank's *Data Mining* for the expectation-maximization (EM) clustering algorithm. That reference is commonly associated with Java implementations, but for our work the important artifact is the algorithm itself, not the language of the original code. Going forward, we want a formulation that can be implemented in Forth.

This document therefore expresses EM clustering in a way that matches Forth development:

- explicit arrays instead of classes,
- small words with clear stack effects,
- loops over samples and clusters,
- simple convergence checks,
- minimal assumptions about runtime support.

Page 17: Histogram Matching

Before the paper reaches the EM appendix, it describes a simpler visual memory on page 17. Each normalized image is reduced to a 16×16 two-dimensional color histogram over the retained green and blue dimensions. The result is a flat vector of 256 values. In the paper's framing, these histograms are not yet full object concepts; they are stored views that capture both the object and its background.

For Forth work, this matters because it is the more immediate candidate for a reusable module. A histogram-based matcher can be implemented independently of the later EM clustering code and can serve as the visual comparison layer for stored views.

The paper compares histograms using equation 1, a χ^2-style divergence:

$$D(h, g) = \sum_{i=1}^{256} \frac{(h_i - g_i)^2}{h_i + g_i} \tag{1}$$

where h_i and g_i are the values of the two histograms in bin i. When a denominator is zero, the corresponding term should be treated as zero in code. The stored view with the smallest pairwise divergence from the input histogram is considered the winning view.

Histogram Module Shape

A practical Forth module for the page 17 mechanism can start with these pieces:

- ‘histogram-bins‘ set to 256,
- a flat float array for each stored histogram,
- an address helper for ‘hist[i]‘,
- ‘hist-divergence (hist-a hist-b – r)‘,
- ‘nearest-view (query views count – best-index)‘.

Suggested stack-oriented words:

- ‘hist-bin-addr (hist i – addr)‘
- ‘hist@ (hist i – r)‘
- ‘hist! (r hist i –)‘
- ‘clear-hist (hist –)‘
- ‘hist-divergence (hist-a hist-b – r)‘
- ‘nearest-view (query view-table count – best-index)‘

Forth-Style Histogram Pseudocode

```
\ hist-divergence ( hist-a hist-b -- r )
\ total = 0
\ for i in 0 .. 255:
\   a = hist-a[i]
\   b = hist-b[i]
\   denom = a + b
\   if denom > 0:
\     d = a - b
\     total += (d * d) / denom
\ return total
;

\ nearest-view ( query views count -- best-index )
\ best-index = 0
\ best-score = +infinity
\ for each stored view j:
\   score = hist-divergence( query, views[j] )
\   if score < best-score:
\     best-score = score
\     best-index = j
\ return best-index
;
```

Algorithm

The appendix later switches to unsupervised clustering. We assume one numeric attribute per sample, matching the simplified appendix. For K clusters and N samples:

- x_i is sample i,
- μ_k is the mean of cluster k,
- σ_k is the standard deviation of cluster k,
- p_{ik} is the probability that sample i belongs to cluster k.

The Gaussian probability density for cluster k is:

$$g(x_i, k) = \frac{1}{\sqrt{2\pi}\sigma_k} \exp\left(-\frac{(x_i - \mu_k)^2}{2\sigma_k^2}\right) \tag{2}$$

The expectation step computes normalized membership values:

$$p_{ik} = \frac{g(x_i, k)}{\sum_{j=1}^{K} g(x_i, j)} \tag{3}$$

The maximization step recomputes the cluster parameters:

$$\mu_k = \frac{\sum_{i=1}^{N} p_{ik} x_i}{\sum_{i=1}^{N} p_{ik}} \tag{4}$$

$$\sigma_k = \sqrt{\frac{\sum_{i=1}^{N} p_{ik}(x_i - \mu_k)^2}{\sum_{i=1}^{N} p_{ik}}} \tag{5}$$

The loop repeats until the change in log-likelihood is below a threshold for a chosen number of iterations.

Forth Data Model

A Forth implementation can store the EM state in flat arrays:

- ‘samples[]‘ for the input values x_i,
- ‘means[]‘ for μ_k,
- ‘sigmas[]‘ for σ_k,
- ‘probs[]‘ for p_{ik} stored as a flattened $N \times K$ table,
- ‘last-log-likelihood‘ and ‘current-log-likelihood‘ as scalars.

An address helper for the probability table is useful:

```
\ p-addr ( sample-index cluster-index -- addr )
\ returns the address of p_{ik} in a flattened table
\ cluster count K is assumed to be available
\ addr = probs + ((i * K) + k) * float-size
```

Suggested Word Set

The implementation can be decomposed into a small set of words:

- ‘gaussian-pdf (x k – r)‘
- ‘expectation-step (–)‘
- ‘update-mean (k –)‘
- ‘update-sigma (k –)‘
- ‘maximization-step (–)‘
- ‘sample-likelihood (i – r)‘
- ‘log-likelihood (– r)‘
- ‘converged? (– flag)‘
- ‘em-iterate (–)‘
- ‘run-em (–)‘

These names are descriptive rather than canonical. In a constrained target, shorter words may be preferred once the behavior is stable.

Forth-Style Pseudocode

```
\ gaussian-pdf ( x k -- r )
\ Use means[k] and sigmas[k]
\ Compute:
\   1 / (sqrt(2*pi) * sigma[k])
\   * exp( -((x - mean[k])^2) / (2 * sigma[k]^2) )
;

\ expectation-step ( -- )
\ for each sample i:
\   denom = 0
\   for each cluster k:
\     tmp[k] = gaussian-pdf( samples[i], k )
\     denom += tmp[k]
\   for each cluster k:
\     probs[i,k] = tmp[k] / denom
;

\ update-mean ( k -- )
\ weighted-sum = 0
\ weight-total = 0
\ for each sample i:
\   p = probs[i,k]
\   weighted-sum += p * samples[i]
\   weight-total += p
\ mean[k] = weighted-sum / weight-total
```

```
;

\ update-sigma ( k -- )
\ variance-sum = 0
\ weight-total = 0
\ for each sample i:
\   p = probs[i,k]
\   d = samples[i] - mean[k]
\   variance-sum += p * d * d
\   weight-total += p
\ sigma[k] = sqrt( variance-sum / weight-total )
;

\ maximization-step ( -- )
\ for each cluster k:
\   k update-mean
\   k update-sigma
;

\ log-likelihood ( -- r )
\ total = 0
\ for each sample i:
\   s = 0
\   for each cluster k:
\     s += gaussian-pdf( samples[i], k )
\   total += log( s )
\ return total
;

\ converged? ( -- flag )
\ abs(current-log-likelihood - last-log-likelihood) < threshold
;

\ run-em ( -- )
\ initialize means and sigmas
\ begin
\   current-log-likelihood -> last-log-likelihood
\   expectation-step
\   maximization-step
\   log-likelihood -> current-log-likelihood
\   converged?
\ until
;
```

Implementation Notes

- Guard against zero or near-zero σ_k. A minimum sigma floor is practical in Forth as well as in higher-level languages.
- If the target Forth lacks floating-point support, fixed-point arithmetic may be possible, but the exponential and logarithm functions will need careful approximation.

- For embedded work, temporary storage for the current sample's unnormalized cluster scores is cheaper than recomputing them during normalization.
- If multiple numeric features are needed later, the one-dimensional Gaussian can be extended, but the memory layout should be revisited before scaling up.

Why This Version Exists

This file is not a translation of Java source. It is a language-shifted technical note intended to bridge both page 17's histogram matcher and the paper's EM appendix into a Forth implementation path. That keeps the algorithms tied to the paper's simplified description while staying aligned with the project's future implementation direction.

References

[1] Ian H. Witten and Eibe Frank. Data Mining. Morgan Kaufmann Publishers, 2000.